REFONTE INTERNATIONAL TRAINING AND INTERNSHIP PROGRAM

Astrodynamics Specialist

SESSION 10

Optimization Methods for Mission Design

Finding the Best Design, Not Just a Working One

Prepared for	Cao Khanh Tan
Module	10, the final taught module before the capstone
Builds on	Sessions 3, 5 and 6 (transfers, constellations, station-keeping)
Tools	Python (scipy.optimize, pymoo), MATLAB, GMAT and STK optimizers

SESSION 10

Optimization Methods for Mission Design

Every module so far has produced designs that work: a transfer that reaches Mars, a constellation that covers the Earth, a station-keeping plan that holds an orbit. This final module asks the next question an engineer always faces: is it the best design? Optimization is the discipline of searching a space of possible designs for the one that minimises cost, or maximises performance, subject to the constraints the mission imposes. It is the thread running through the whole course, and the tool you will lean on most in the capstone.

Learning objectives

- State an optimization problem in standard form: objective, design variables and constraints.
- Find an optimum for a simple design problem, analytically and numerically.
- Explain multi-objective optimization and read a Pareto front.
- Match the main optimization methods to the kinds of problem they suit.
- Recognise where earlier modules were really optimizations in disguise.
- Set up and solve a mission-design optimization in code.

> **The shift in mindset**
> A working design answers "does it meet the requirement?" An optimal design answers "what is the least it can cost while meeting the requirement, and what would it cost to do better?" The second question is what turns analysis into engineering.

PART 1

The Optimization Framework

Every optimization has the same three ingredients. The objective (or cost) function J is the single number you want to make small: total delta-v, satellite count, propellant mass, transfer time. The design variables x are the knobs you may turn: burn sizes and times, orbital elements, the number of planes, a launch date. The constraints fence off the feasible designs: equalities that must hold (arrive at the target) and inequalities that must not be violated (a delta-v budget, a maximum g-load, a minimum elevation). In standard form:

$$\min_{\mathbf{x}} J(\mathbf{x}) \quad \text{s.t.} \quad \mathbf{g}(\mathbf{x}) \leq 0, \quad \mathbf{h}(\mathbf{x}) = 0$$

At an unconstrained optimum the gradient of the objective vanishes; with constraints, the Lagrangian combines the objective and the constraints so that the same stationarity condition applies to it:

$$\nabla J(\mathbf{x}^*) = 0$$

$$\mathcal{L} = J(\mathbf{x}) + \boldsymbol{\lambda}^T \mathbf{h}(\mathbf{x}) + \boldsymbol{\mu}^T \mathbf{g}(\mathbf{x})$$

Getting the problem right matters more than the method

Most optimization failures are not solver failures; they are a poorly posed objective, a missing constraint, or design variables that cannot actually express a good solution. Write the three ingredients down explicitly before reaching for a tool.

PART 2

A Worked Optimum: Splitting a Plane Change

Optimization often finds an answer intuition would miss. Take the LEO-to-GEO transfer that must also remove a 28.5 degree launch inclination (Session 3). The plane change can be done at perigee, at apogee, or split between the two. Session 3 said "do it at apogee, where you are slowest". Optimization sharpens that: let the design variable be the fraction of the plane change done at perigee, and minimise the total delta-v.

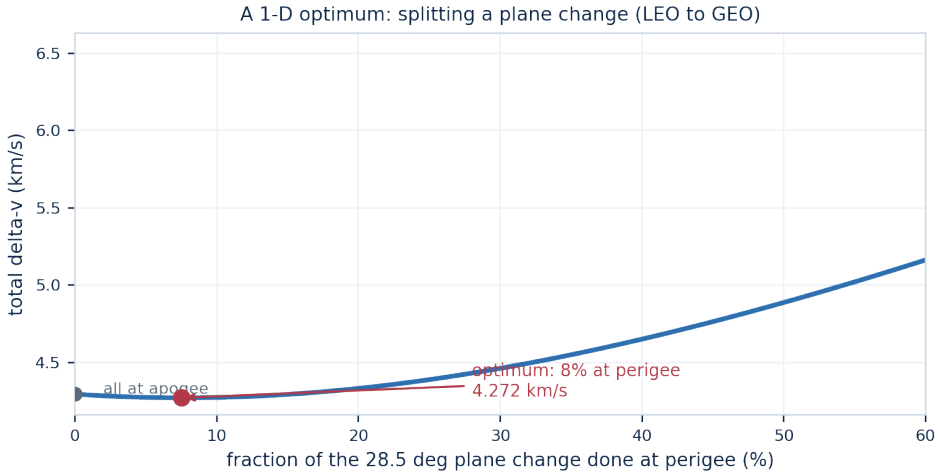


Figure 1. Total delta-v against the fraction of the plane change done at perigee. The minimum is not at zero: doing a small part (about 8 percent) of the turn during the first burn, when it can be combined with the apogee-raising, is cheaper than doing it all at apogee, saving about 24 m/s.

The lesson is general. The cheapest design sits at an interior point that no rule of thumb would have named; only writing down the cost function and searching it reveals it. For a one-variable problem a plot or a line search is enough; real problems need the methods of Part 4.

PART 3

Many Objectives at Once: Pareto Fronts

Mission design rarely has a single objective. A constellation should have few satellites (cheap) and short revisit (good), but these fight each other. When objectives conflict there is no single best design; instead there is a set of best trade-offs, the Pareto front, where you cannot improve one objective without worsening another:

$$\mathbf{x}_1 \text{ dominates } \mathbf{x}_2 : \; J_i(\mathbf{x}_1) \leq J_i(\mathbf{x}_2) \; \forall i$$

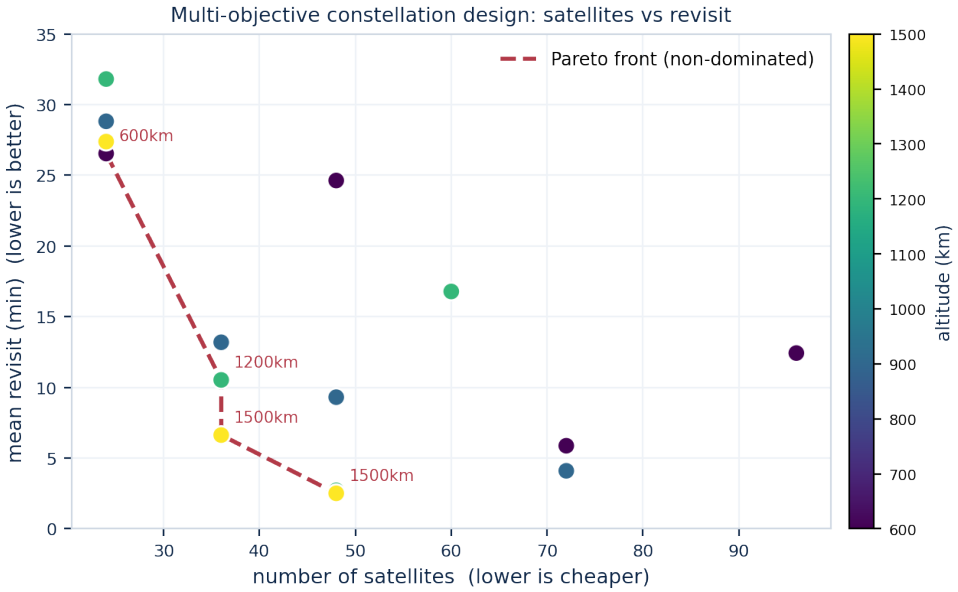


Figure 2. A real trade study: fifteen LEO constellations scored on satellite count and mean revisit. The dashed red line is the Pareto front, the designs that are not beaten on both counts at once. Everything above and to the right of it is wasteful; the designer picks a point on the front according to budget and revisit needs.

Reading a Pareto front

A point off the front can be improved for free in at least one objective, so it is never the right choice. Points on the front are all defensible; choosing among them is a value judgement, not a calculation. This is exactly the trade study you did in the Session 5 constellation project, now named for what it is.

PART 4

The Methods

There is no universal optimizer; the right method depends on the problem. The main families, and where they fit:

Method	How it works	Best for
Analytic	Set the gradient to zero and solve	Simple, smooth problems (the Hohmann optimum)
Gradient-based (SQP)	Follow the slope to a local optimum	Smooth problems with derivatives; fast, local
Direct transcription	Discretise the trajectory into a nonlinear program	Low-thrust and continuous-control trajectories
Global / heuristic (GA)	Search broadly with populations, no gradient	Rugged, multi-modal spaces; gravity-assist tours
Grid / parametric sweep	Evaluate every point on a grid	Small design spaces; porkchop launch windows

Gradient methods are fast but only find the nearest valley, which is a trap when the design space is rugged, as many astrodynamics problems are:

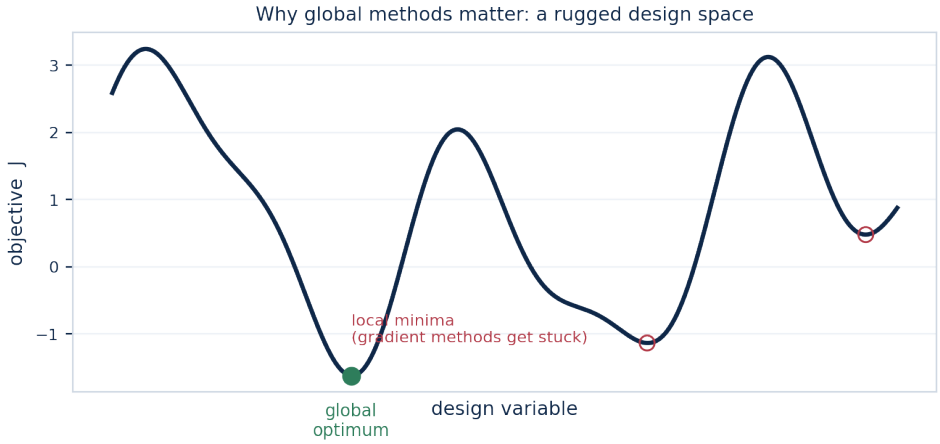


Figure 3. A rugged objective with several local minima. A gradient method started at the wrong place settles into a local minimum (red) and reports success; only a global method, or many restarts, reliably finds the true optimum (green). Launch-window and gravity-assist searches look like this.

PART 5

The Whole Course Was Optimization

Seen through this lens, much of the programme was optimization in disguise:

- **The Hohmann transfer (S3)** is the minimum-delta-v two-burn transfer between coplanar circular orbits, an analytic optimum.
- **Sun-synchronous and critical inclinations (S2)** are the specific values that make a perturbation do exactly what the mission needs.
- **The constellation trade study (S5)** was a search for the Pareto-optimal satellite count against revisit.
- **Lifetime station-keeping (S6)** minimises propellant subject to holding a dead-band.
- **Launch windows and porkchop plots (S4)** are a grid search for the minimum-energy departure date.

Naming these as optimizations is not just tidy; it tells you which method to reach for when the capstone asks you to do better than a first guess.

Solving an Optimization in Code

The plane-change split is a one-line call to a numerical optimizer. The same pattern (define the objective, hand it to a solver) scales to far harder problems.

```
import numpy as np
from scipy.optimize import minimize_scalar
mu, Re = 398600.4418, 6378.137
vc  = lambda r: np.sqrt(mu/r)
vis = lambda r,a: np.sqrt(mu*(2/r - 1/a))
r1, r2, di = Re+185, 42164.0, np.radians(28.5)
a_t = (r1+r2)/2
vleo, vp, va, vgeo = vc(r1), vis(r1,a_t), vis(r2,a_t), vc(r2)

def total_dv(f):                                    # f = plane change at perigee
    dv1 = np.sqrt(vleo**2 + vp**2   - 2*vleo*vp*np.cos(f*di))
    dv2 = np.sqrt(va**2   + vgeo**2 - 2*va*vgeo*np.cos((1-f)*di))
    return dv1 + dv2

res = minimize_scalar(total_dv, bounds=(0,1), method='bounded')
print(res.x, res.fun)          # ~0.076, ~4.272 km/s
```

Production tools

Python (scipy.optimize for local, pymoo or pygmo for multi-objective and global), MATLAB Optimization Toolbox, and the optimizers built into GMAT (its Optimize sequence with a solver like VF13ad) and STK Astrogator all solve these problems. The method matters less than a well-posed objective and honest constraints.

PRACTICE

This Week (aim for 12 to 15 hours)

Bring your work and plots to the next live session. As always, show your reasoning, not just the final numbers.

Assignment: Optimization Methods for Mission Design

- **Reading and notes (3 to 4 h).** Write an optimization problem you have met in this course in standard form: objective, design variables and constraints. Explain what a Pareto front is and why a point off it is never chosen. One page.
- **One-variable optimum (3 to 4 h).** Repeat the plane-change split for a 51.6 degree launch inclination instead of 28.5. Find the optimal fraction done at perigee, the minimum total delta-v, and the saving over doing it all at apogee.
- **Pareto reasoning (3 to 4 h).** From the constellation trade in Figure 2, choose a design for a mission requiring a mean revisit under 8 minutes at the lowest satellite count, and justify it. Then choose differently if the priority is the fewest satellites above 90 percent coverage.
- **Method matching (2 h).** For each of these, name a suitable method and say why: an Earth-to-Mars launch window; a low-thrust spiral trajectory; sizing a Walker constellation; a gravity-assist tour of the outer planets.
- **Code (2 h).** Using scipy.optimize (or MATLAB), reproduce the Task 2 optimum and confirm it numerically.

Answer key (for your self-check)

Quantity	Expected value
Plane-change split at 51.6 deg	optimum about 5.4 percent at perigee; total about 4.85 km/s
Saving vs all-at-apogee (51.6 deg)	about 39 m/s (all-at-apogee about 4.89 km/s)
Launch window / porkchop	grid or global search (many local minima over dates)
Low-thrust trajectory	direct transcription / collocation to a nonlinear program
Constellation sizing	multi-objective global (genetic algorithm) over Walker parameters

REFERENCE

One-Page Formula Sheet

Standard form	$\min_{\mathbf{x}} J(\mathbf{x}) \quad \text{s.t.} \quad \mathbf{g}(\mathbf{x}) \leq 0, \quad \mathbf{h}(\mathbf{x}) = 0$
Unconstrained optimum	$\nabla J(\mathbf{x}^*) = 0$
Lagrangian (constrained)	$\mathcal{L} = J(\mathbf{x}) + \boldsymbol{\lambda}^T \mathbf{h}(\mathbf{x}) + \boldsymbol{\mu}^T \mathbf{g}(\mathbf{x})$
Gradient descent step	$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla J(\mathbf{x}_k)$
Pareto dominance	$\mathbf{x}_1$ dominates $\mathbf{x}_2$: $J_i(\mathbf{x}_1) \leq J_i(\mathbf{x}_2)\ \forall i$

Key glossary

- **Objective (cost) function:** the single number the optimizer minimises or maximises.
- **Design variables:** the quantities the optimizer is free to change.
- **Feasible region:** the set of designs that satisfy every constraint.
- **Pareto front:** the set of best trade-offs when objectives conflict.
- **Local vs global optimum:** the best nearby point vs the best point anywhere in the space.

Going further

For depth, see Betts, Practical Methods for Optimal Control and Estimation (trajectory optimization), Nocedal and Wright, Numerical Optimization (the methods), and Vallado for the astrodynamics applications. These are real, widely used texts; please confirm editions yourself, as I cannot verify those details from here.

Coming up: the Capstone

You now have the full astrodynamics toolkit and the optimization mindset to use it well. The final module is the capstone: choose a mission, design it, optimise it, simulate it, and defend it. Everything from the two-body problem to this page comes together there.